Databricks Hackathon 2026

GlobalMart - Retail Use Case

Architecture Document — by 4Ceers

Arnab Ray
Kushagra Verma
Pranjal Sancheti
Pavan V

Table of Contents

Table of Contents 2
1. Business Mandate 3
1.1 Goals and outcomes 3
2. Source Data Landscape 4
3. Platform Architecture 5
3.1 Governance Layer — Unity Catalog 5
3.2 Catalog and Schema Architecture: 6
3.3 Data flow diagram 7
3.4 Data Layers — Medallion Architecture 9
3.5 Pipeline Design — Spark Declarative Pipelines (SDP) 9
3.6 Component Map 11
3.7 Analytical Model - Star Schema 12
3.8 Serving Layer 16
4. Gen AI Intelligence Layer 17
5. Generative AI Integration 19
Model and access pattern 19
Overview of Usecases 20
UC1: AI Data Quality Reporter 20
UC2: Returns Fraud Investigator 21
UC3: Product Intelligence Assistant (RAG) 21
UC4: Executive Business Intelligence 22
Quality validation - all use cases 24
Design principles 24
5. Data Quality Strategy 24
5.1 Quality Enforcement Across Layers 25
5.2 Integration with the Broader Architecture 25

1. Business Mandate

GlobalMart operates across 5 US regions (East, West, Central, South, North) with 6 acquired regional systems. Three simultaneous failures triggered this initiative:

Failure	Business Impact	Root Cause	Deadline / Exposure
Revenue Audit Failure	Overlapping customer orders under different IDs overstated revenue by ~9%. Quarterly reconciliation takes 3 days, wrong 30% of the time.	No unified order registry; no lineage to source files.	60 days to unified, reconcilable order registry or face annual filing qualification risk.
Returns Fraud Exposure	$2.3M annual loss. High-value returns verified across 3 disconnected systems; 2 weeks per case. Agents see no cross-region history.	Fragmented customer return history; no cross-system view.	$2.3M/year ongoing loss; undetected until financial close.
Inventory Blindspot	12–18% potential revenue lost from slow-moving inventory never flagged for redistribution or discount. Product surpluses discovered only at quarterly review.	No shared visibility across regions; no shared inventory velocity metric.	$2.1M slow-moving inventory exposure; W region worst at $698K.

1.1 Goals and outcomes

- Data from 6 disconnected regional systems can be unified and trusted automatically, without manual patching.
- Data quality issues are caught, quarantined, and explained—not silently ignored.
- Business users can ask questions and get answers without writing SQL or waiting for engineering.
- AI can explain data health and surface insights in language that leadership understands.

2. Source Data Landscape

16 raw files across 6 entity groups, ingested from 6 regional sub-folders. Total volume: ~16,151 rows.

Entity	Tables	Rows	Critical Schema Issues
Customers	6 CSV	748	4 different PK names (customer_id, CustomerID, cust_id, customer_identifier). customers_5 missing email column. customers_6 renames every column. customers_4 swaps city/state column order.
Orders	3 CSV	5,006	orders_3 missing order_estimated_delivery_date. All dates stored as strings (MM/DD/YYYY HH:MM). ship_mode 11.4% null in orders_2. ? placeholders in carrier dates.
Transactions	3 CSV	9,816	transactions_2 missing profit column. 20.5% of transactions_1.Sales prefixed with $. Column casing varies (Order_id vs Order_ID). 9.6% of payment_type = ?.
Returns	2 JSON	800	Every column is renamed between tables (order_id/OrderId, refund_amount/amount, return_date/date_of_return). Abbreviations in return_status (PENDG, APPRVD, RJCTD). 16.6% of returns_1.refund_amount prefixed with $.
Products	1 JSON	1,774	High sparsity: weight (94.4% null), dimension (87.6%), sizes (65.6%), manufacturer (59.0%), upc (39.0%). Dates in ISO 8601.
Vendors	1 CSV	7	Clean. Only 4 of 7 vendors referenced in orders (VEN01–VEN04).

Referential integrity is fully intact: zero orphaned foreign keys across all entity relationships. 332 of 748 customers (44.4%) have placed orders. All 1,774 products appear in transactions. ***However, returns are traceable at an order level, not product level****.*

3. Platform Architecture

3.1 Governance Layer — Unity Catalog

Unity Catalog is the single governance plane for file storage, access control, and data lineage across the entire platform.

Capability	Implementation
Catalog Structure	globalmart.bronze.* | globalmart.silver.* | globalmart.gold.* — three-level namespace (catalog.schema.table) governs all objects.
Discovery	Every table registered in Unity Catalog with schema metadata.
Lineage	Automatic column-level lineage from source file → Bronze → Silver → Gold. Every revenue figure traces to the exact source CSV/JSON via source_file and load_timestamp columns.
Volume Storage	Raw files land in /Volumes/globalmart/raw/source_files/ organized by region sub-folders. Unity Catalog Volumes manage file-level governance.

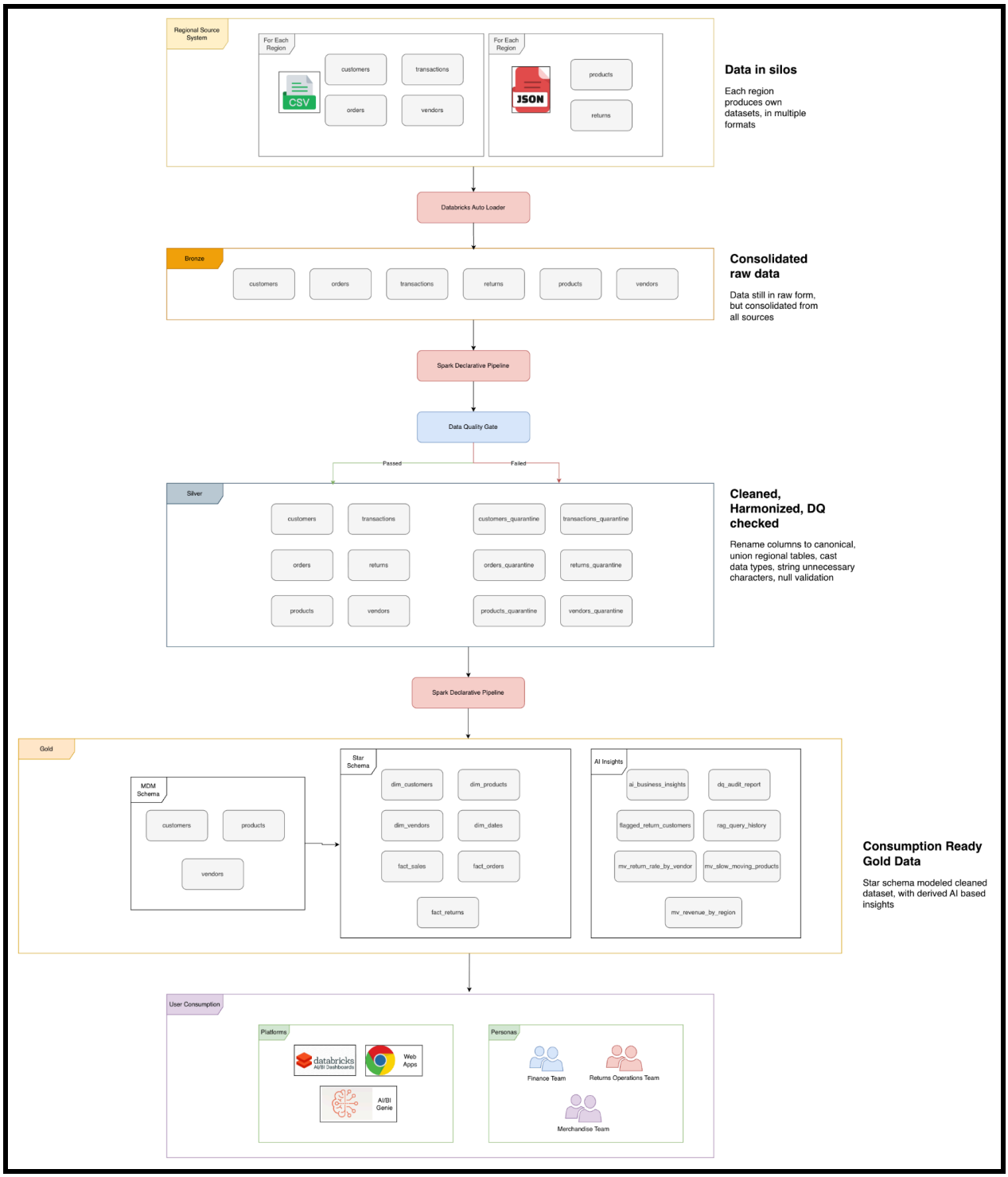
Design rationale: The current state has no lineage. When auditors ask which file a revenue figure came from, no one has a traceable answer. Unity Catalog's automatic lineage tracking directly resolves this audit failure.

3.2 Catalog and Schema Architecture:

```
Unity Catalog (Catalog -> Schema -> Table)
+-- globalmart
|  +-- config
|     +-- column_mapping
|     +-- dq_rules
|     +-- fraud_rule_config
|     +-- mdm_rules
|     +-- pipeline_run_log
|  +-- raw
|     +-- schema_validation_bronze
|     +-- source_files (Volume)
|  |  |                    +-- Region 1…6 (Folder)
|  |  |                    |          +-- customers
|  |  |                    |          +-- orders (if exists in the region)
|  |  |                    |          +-- transactions (if exists in the region)
|  |  |                    |          +-- returns (if exists in the region)
|  |  |                    +-- products (File)
|  |  |                    +-- vendors (File)
|  +-- bronze
|  |  +-- customers
|  |  +-- orders
|  |  +-- transactions
|  |  +-- returns
|  |  +-- products
|  |  +-- vendors
|  +-- silver
|  |  +-- customers
|  |  +-- orders
|  |  +-- transactions
|  |  +-- returns
|  |  +-- products
|  |  +-- vendors
|  |  +-- customers_quarantine
|  |  +-- orders_quarantine
|  |  +-- transactions_quarantine
|  |  +-- returns_quarantine
|  |  +-- products_quarantine
|  |  +-- vendors_quarantine
|     +-- mdm
|     +-- customers
|     +-- products
|     +-- vendors
|  +-- gold
|  |  +-- fact_orders
|  |  +-- fact_sales
|  |  +-- fact_sales_quarantine
|  |  +-- fact_returns
|  |  +-- fact_returns_quarantine
|  |  +-- dim_customers
|  |  +-- dim_products
|  |  +-- dim_vendors
|  |  +-- dim_dates
|  |  +-- ai_business_insights
|  |  +-- dq_audit_report
|  |  +-- gold_dq_audit_report
|  |  +-- flagged_return_customers
```

```
|   |   +-- rag_query_history
|   |   +-- mv_return_rate_by_vendor
|   |   +-- mv_return_rate_by_vendor_quarantine
|   |   +-- mv_slow_moving_products
|   |   +-- mv_slow_moving_products_quarantine
|   |   +-- mv_revenue_by_region
|   |   +-- mv_revenue_by_region_quarantine
|   |   +-- mv_customer_return_history
|   |   +-- customer_id_crosswalk
|   |   +-- metricview_inventory
|   |   +-- metricview_revenue
|   |   +-- metricview_vendor_returns
```

3.3 Data flow diagram


Regional Source System
For Each Region
customers
transactions
orders
vendors
CSV
For Each Region
products
returns
JSON
Data in silos
Each region produces own datasets, in multiple formats
Databricks Auto Loader
Bronze
customers
orders
transactions
returns
products
vendors
Consolidated raw data
Data still in raw form, but consolidated from all sources
Spark Declarative Pipeline
Data Quality Gate
Passed
Failed
Silver
customers
transactions
customers_quarantine
transactions_quarantine
orders
returns
orders_quarantine
returns_quarantine
products
vendors
products_quarantine
vendors_quarantine
Cleaned, Harmonized, DQ checked
Rename columns to canonical, union regional tables, cast data types, string unnecessary characters, null validation
Spark Declarative Pipeline
Gold
MDM Schema
customers
products
vendors
Star Schema
dim_customers
dim_products
dim_vendors
dim_dates
fact_sales
fact_orders
fact_returns
AI Insights
ai_business_insights
dq_audit_report
flagged_return_customers
rag_query_history
mv_return_rate_by_vendor
mv_slow_moving_products
mv_revenue_by_region
Consumption Ready Gold Data
Star schema modeled cleaned dataset, with derived AI based insights
User Consumption
Platforms
databricks
AI/BI Dashboards
Web Apps
AI/BI Genie
Personas
Finance Team
Returns Operations Team
Merchandise Team

3.4 Data Layers — Medallion Architecture

Layer	Entry State	Exit State	Format
Bronze	Raw CSV/JSON from 6 regional systems. All columns STRING. Schema varies per source (4 different PK names, missing columns, swapped columns).	Exact copy of source + source_file, load_timestamp, rescued_data columns appended. Schema validation results written to schema_validation_bronze.	Delta tables (append-only)
Silver	Raw Bronze records with inconsistent names, types, formats, placeholders, and missing fields.	Harmonized column names. Proper data types (DECIMAL, TIMESTAMP, DATE). Placeholders replaced with NULL. $ prefixes stripped. Abbreviations standardized. Clean records only.	Delta tables (versioned, merge-capable)
Gold	Harmonized Silver tables: unified customers, orders, transactions, returns, products, vendors.	Star schema: dim_customers, dim_products, dim_vendors, dim_dates, fact_sales, fact_returns. Plus MVs, AI output tables, and Metric Views.	Delta tables + Metric View

Design rationale: The current state rebuilds all tables from scratch every run. The medallion architecture with Delta tables enables incremental processing via MERGE, time-travel for audit queries, and schema enforcement at each layer boundary.

3.5 Pipeline Design — Spark Declarative Pipelines (SDP)

The platform's data movement and transformation are orchestrated entirely via Spark Declarative Pipelines (SDP). The architecture has transitioned from static, hardcoded scripts to a Dynamic Metadata-Driven model. This ensures that as new regional files are acquired or schemas drift, the pipeline adapts automatically without manual code deployments.

Pipeline Flow

Step	Stage	Action	SDP Behaviour

1	**Pre-Bronze**	Contract Validation	schema_validation_bronze acts as a guardrail. It deterministically evaluates incoming Bronze streams against required min_cols and key_variants. Entities missing fundamental identifiers trigger a WARN status before Silver processing begins.
2	**Bronze**	Raw Ingestion	Uses Databricks Auto Loader (cloudFiles) for incremental file ingestion from Unity Catalog Volumes. schemaEvolutionMode: addNewColumns prevents pipeline failures from regional schema drift. Lineage (source_file, load_timestamp) is permanently stamped on every record.
3	**Silver**	Schema Discovery	An LLM Schema Mapper dynamically profiles Bronze data and calls databricks-gpt-oss-20b. It semantically maps disparate regional column names to the target Canonical Silver schema and generates necessary SQL casting transformations.
4	**Silver**	Harmonization	The harmonization engine executes the LLM-generated mappings (with a static config fallback). It applies COALESCE to merge variant columns (e.g., cust_id and customer_identifier), standardizes data types, and sanitizes strings.
5	**Silver**	DQ & Quarantine	Evaluates deterministic data quality rules fetched from globalmart.ingestion_control. @dp.expect_or_drop prevents critical failures from advancing. Failing rows are tagged with rule metadata and seamlessly routed to entity-specific quarantine tables without halting the pipeline.
6	**MDM**	Entity Resolution	Applies MDM engine rules to resolve conflicting cross-regional data (e.g., 6 different customer lists). Logic dictates source priority, most-recent timestamps, or most-complete data to generate a single "Golden Record" per entity.
7	**Gold**	Aggregation & Joins	Uses dp.read() for intra-pipeline dependencies to build the Star Schema. Generates Dimension tables, Fact tables (Streaming with identity translation), and Materialized Views. customer_id_crosswalk is generated inside the Gold pipeline to guarantee dependency resolution.
8	**Gold**	Business DQ	Enforces advanced business logic via Expectations at the top of the stack. Identifies "Orphan Customers", mathematically impossible return rates,

			and "Returns Fraud" (refunds > 5x sales). Failures are routed to Gold quarantine tables for immediate business review.

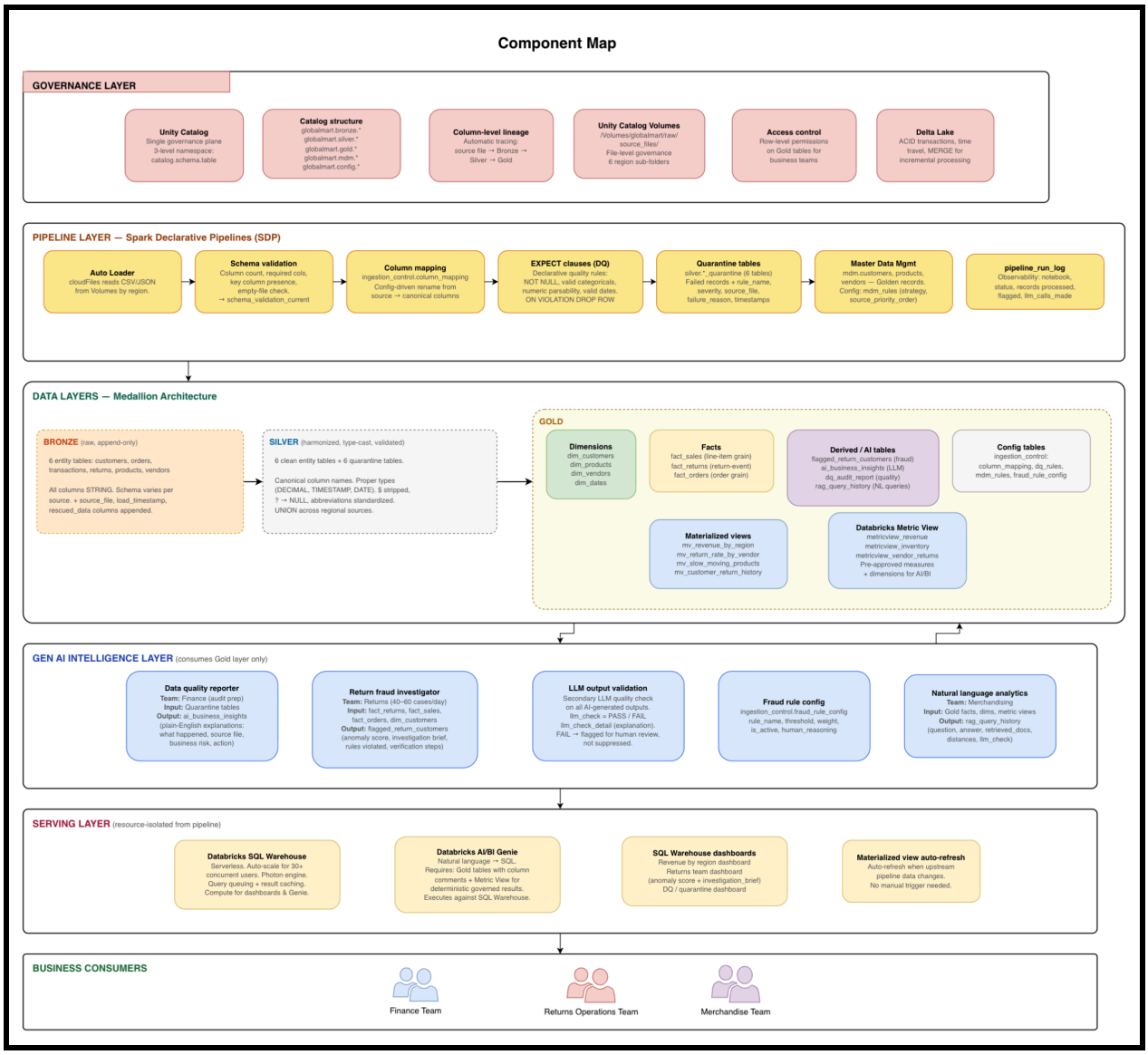
Quarantine Architecture

Our expectation strategy follows two principles:

- No record is ever permanently lost. Every record that fails any expectation — whether critical or advisory — is captured in a quarantine table for that entity. Critical failures are removed from the main Silver table. Advisory failures remain in the main table but are also logged in quarantine for visibility.
- Severity-based triage. Each expectation is tagged with a severity level that determines the outcome and guides downstream investigation by the finance and returns teams.
- Each quarantined record is tagged with: _expectation_name, _severity, _issue_type, _failure_reason, and _run_id. Gold quarantine tables additionally carry a _business_impact column explaining the downstream consequence in plain English.

Design rationale: The current state has no shared quality checks. Bad data is discovered in quarterly reports or audits—not at ingestion. SDP’s declarative quality expectations enforce rules at pipeline runtime, quarantine failures for investigation, and continue processing without stopping—directly resolving the audit reconciliation failure.

3.6 Component Map



3.7 Analytical Model - Star Schema

Gold layer tables follow a star schema at order-line-item grain for sales and at return-event grain for returns. All cross-entity business questions (revenue by region, return rate by vendor, profitability by customer segment) resolve to single, consistent queries against this model.

Dimension Tables

Table	Grain	Key Columns

dim_customers	1 row per customer	customer_key (surrogate), customer_id, customer_name, customer_email, segment, country, city, state, postal_code, region
dim_products	1 row per product	product_key (surrogate), product_id, product_name, brand, categories, colors, manufacturer
dim_vendors	1 row per vendor	vendor_key (surrogate), vendor_id, vendor_name
dim_dates	1 row per calendar day	date_key (YYYYMMDD), full_date, year, quarter, month, month_name, day_of_week, day_name, is_weekend

Fact Tables

Table	Grain	Key Columns + Measures
fact_orders	1 row per order	order_key, order_id, customer_id, vendor_id, ship_mode, order_status, order_purchase_timestamp, order_approved_timestamp, order_delivered_carrier_timestamp, order_delivered_customer_timestamp. FK: customer_id → dim_customers, vendor_id → dim_vendors.
fact_sales	1 row per order line item	sales_key, customer_key, customer_id, product_key, product_id, vendor_key, vendor_id, order_date_key, order_id, sales, quantity, discount, profit, payment_type, payment_installments. FK: all surrogate keys → dimensions. Source: silver.transactions joined to silver.orders.
fact_returns	1 row per return event	return_key, customer_key, customer_id, product_key, product_id, vendor_key, vendor_id, return_date_key, order_id, refund_amount, return_reason, return_status, return_date, original_sales, return_to_sales_ratio. Product attribution uses an array-sort lambda that matches refund_amount to the closest line-item sales value from the originating order.

Derived / Analytical Tables

Table	Purpose	Business Failure Resolved

customer_id_crosswalk	Maps every regional customer_id to a canonical winner_customer_id using multi-factor identity fingerprinting. Generated inside the Gold pipeline to guarantee dependency resolution.	Revenue Audit: eliminates the 9% over-counting caused by the same customer having different IDs across 6 regional systems.
mv_revenue_by_region	Monthly revenue and profit aggregated by region. Columns: year, month, month_name, region, total_sales, total_profit, order_count. DQ expectation: region IS NOT NULL.	Revenue Audit: provides a single reconcilable regional revenue view auditors can trace to source files.
mv_return_rate_by_vendor	Return rate per vendor. Columns: vendor_id, vendor_name, total_orders_sold, total_returns, total_refund_amount, return_rate_pct (0–100 scale).	Returns Fraud: identifies vendors with structurally high return rates for procurement action.
mv_slow_moving_products	Days-since-last-sale per product per region. Columns: product_id, product_name, region, last_sale_date, total_quantity_sold, total_sales, days_since_last_sale.	Inventory Blindspot: flags products unsold for 90+ days for discount or redistribution.
mv_customer_return_history	Per-customer return behaviour summary. Columns: customer_id, customer_name, region, total_returns, total_refund_amount, avg_return_ratio, fraud_risk_score (HIGH / MEDIUM / LOW).	Returns Fraud: pre-computed risk score per customer without requiring a real-time join across three tables.
flagged_return_customers	Fraud detection output: anomaly_score (0–100), rules_violated (pipe-delimited), investigation_brief (AI-generated), llm_check. One row per flagged customer per run.	Returns Fraud: eliminates the 2-week manual cross-system investigation. Each flagged customer has a pre-built investigation brief.
dq_audit_report	AI-generated plain-English explanations of Silver quarantine events. One row per issue group (entity + issue_type). Columns: entity, field_affected, issue_type,	Revenue Audit: finance team gets audit-ready explanations without reading Python logs.

	rejected_count, severity, ai_explanation, llm_check.	
gold_dq_audit_report	AI-generated explanations for Gold-layer DQ rule failures. Covers fact_sales, fact_returns, and MV quarantine tables.	All three business failures: Gold-layer quality gaps explained in plain English.
ai_business_insights	LLM-generated executive summaries for three domains: revenue_performance, vendor_return_rate, slow_moving_inventory. Columns: insight_type, ai_summary, kpi_json, llm_check.	Inventory Blindspot + Revenue Audit: leadership gets AI-interpreted summaries with root-cause and action recommendations.
rag_query_history	Log of all RAG queries. Columns: question, answer, retrieved_docs (array), distances, llm_check, queried_at.	Inventory Blindspot: audit trail for all natural language product queries answered by the RAG system.
metricview_revenue	Databricks Metric View on fact_sales. Dimensions: Region, Segment, Year, Quarter, Month, Payment Type. Measures: Total Revenue, Total Profit, Profit Margin Pct, Order Count, Units Sold, Avg Order Value.	Revenue Audit: governed, pre-approved revenue measures for Genie natural language queries.
metricview_vendor_returns	Databricks Metric View on fact_returns. Dimensions: Vendor, Return Reason, Return Status, Return Year. Measures: Total Returns, Total Refund Amount, Avg Refund Amount, Return Rate Pct.	Returns Fraud: governed vendor return measures for Genie natural language queries.
metricview_inventory	Databricks Metric View on mv_slow_moving_products. Dimensions: Region, Brand, Category, Slow Moving Flag. Measures: Slow Product Count, Avg Days Since Last Sale, Total Quantity Sold, Total Sales.	Inventory Blindspot: enables natural language queries on inventory velocity directly from Databricks AI/BI Genie.

How Business Questions Resolve

- "Which products generated the most revenue last quarter?" → fact_sales JOIN dim_products JOIN dim_dates WHERE quarter = N, GROUP BY product, ORDER BY SUM(sales) DESC.
- "Which vendors had the highest return rate?" → mv_return_rate_by_vendor ORDER BY return_rate_pct DESC.
- "Which customer segment is most profitable by region?" → fact_sales JOIN dim_customers, GROUP BY segment, region, SUM(profit).

All queries operate against a single consistent star schema—no cross-system joins, no CSV downloads, no manual reconciliation.

3.8 Serving Layer

Materialised Views + SQL Warehouse

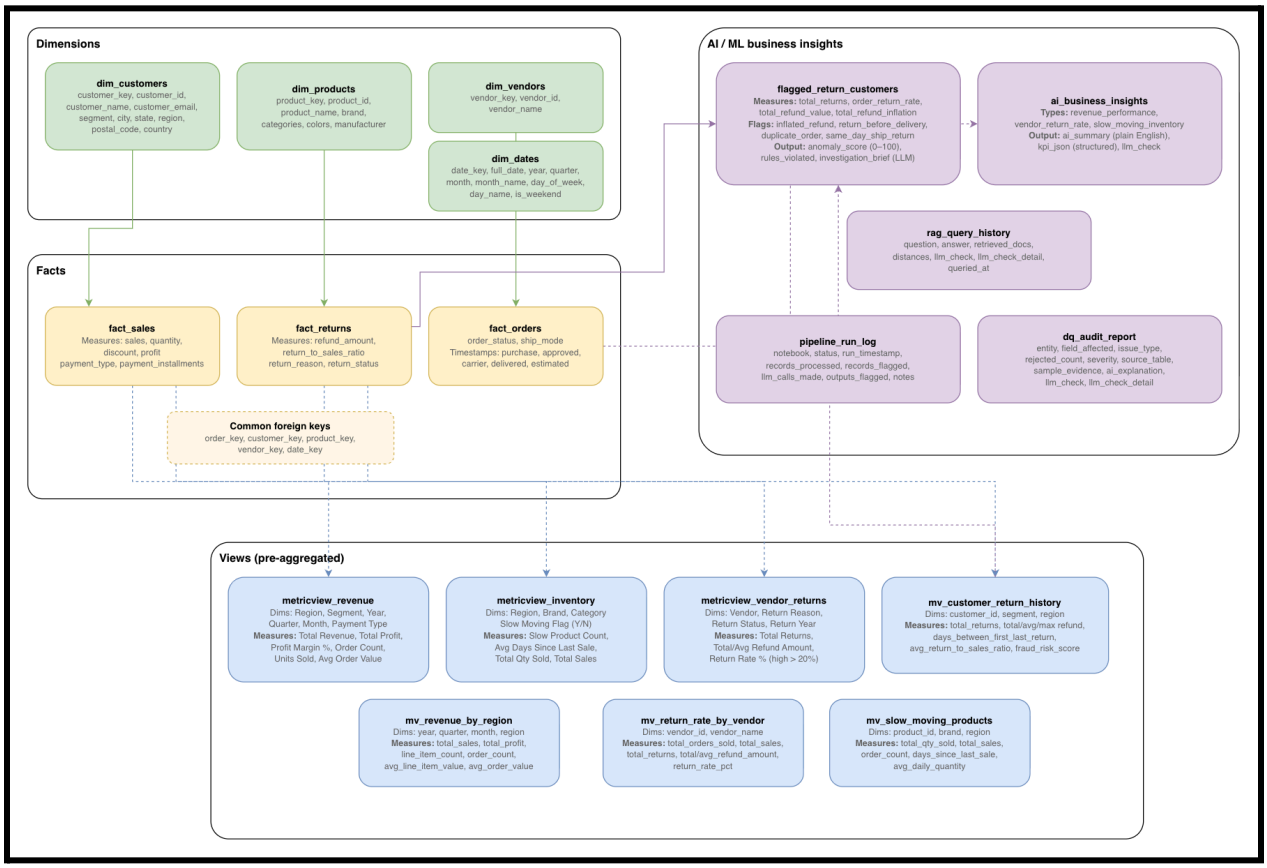
Two components work together to serve the dashboarding requirement:

Component	Role
Materialized Views (Gold layer)	Pre-aggregate revenue by region, return rate by vendor, and slow-moving inventory metrics. Re-computed on each pipeline run as @dp.table definitions within the SDP pipeline. No manual trigger required.
Databricks SQL Warehouse	Dedicated serverless SQL Warehouse serves dashboard queries. Scales independently to handle 30+ concurrent users. Photon engine accelerates aggregation queries on Gold layer Delta tables.
Databricks AI/BI Dashboards	Four published dashboards, one per use case: UC1 DQ Audit, UC2 Fraud Investigator, UC3 Product Intelligence, UC4 Executive BI. All read from Gold layer Delta tables via SQL Warehouse.
Databricks AI/BI Genie	Natural language query interface for the merchandising team. Genie Space includes fact_sales, fact_returns, dim_products, dim_vendors, dim_customers, and all three Metric Views. Translates plain-English questions to SQL executed against the SQL Warehouse.
Streamlit Application	Unified front-end (app.py) connecting all four use cases. Connects to SQL Warehouse via Databricks SDK for data queries and to the Genie Space API for natural language queries. Pages: Home (platform stats), UC1 Data Quality, UC2 Fraud Investigator, UC3 Product Intelligence, UC4 Executive BI & Leads. Reads from: dq_audit_report, flagged_return_customers, rag_query_history, ai_business_insights, mv_slow_moving_products, customer_lead_scores, pipeline_run_log.

Why the Serving Layer Is Separate from the Pipeline

- Resource isolation: SDP pipeline clusters run heavy transformations. Dashboard users querying the same cluster would degrade both pipeline and dashboard performance.
- Cost model: SQL Warehouse auto-scales and auto-suspends. The pipeline cluster runs on a fixed schedule. Coupling them wastes compute.
- Concurrency: SQL Warehouses are engineered for concurrent BI queries (query queuing, result caching, Photon). SDP clusters are engineered for sequential data transformation.

3.9 Analytics layer structure



4. Gen AI Intelligence Layer

The AI layer sits above the Gold layer and consumes only Gold-layer tables. It addresses three specific business needs with three distinct solutions.

Why Gold Layer Only

- Bronze contains raw, ungoverned, untyped data — AI models operating on it would produce unreliable outputs based on $ prefixes, ? placeholders, and inconsistent schemas.

- Silver contains harmonized but not yet modelled data — relationships between entities are implicit, not materialised into a queryable star schema.
- Gold contains validated, reconciled, business-ready data with enforced types, surrogate keys, and computed metrics. AI outputs are only as trustworthy as the data they consume.
- UC1 is the exception: it reads Silver quarantine tables because its job is specifically to explain Silver-layer rejections to the finance auditor.

Use Case	Business Team	Gold Tables Consumed	AI Output	Delivery Channel
Data Quality Reporter	Finance (audit prep)	ai_business_insights (quarantine summaries with insight_type, ai_summary, kpi_json, llm_check)	Plain-English explanation of why specific records were rejected: what happened, which source file, business risk, and recommended action. No stack traces or row counts.	gold.ai_business_insights table. Queryable via SQL Warehouse dashboard or Databricks AI/BI.
Return Fraud Investigator	Returns team (40–60 cases/day)	flagged_return_customers (consumes fact_returns, fact_sales, fact_orders, dim_customers)	Per-customer investigation brief: anomaly score, rules violated (refund inflation, return-before-delivery, duplicate orders, etc.), possible innocent explanation, specific verification steps.	gold.flagged_return_customers table. Anomaly score + investigation_brief columns accessible via returns team dashboard on SQL Warehouse.
Natural Language Analytics	Merchandising team	fact_sales, fact_returns, dim_products, dim_vendors, dim_customers, dim_dates, metricview_inventory	Direct answers to questions like "which products are slow-moving in the West region?" or "which vendors have the highest return rate this quarter?" without SQL.	Databricks AI/BI Genie. Requires Gold tables with descriptive column comments and a Metric View (metricview_inventory) defining business metrics and dimensions.

AI/BI Genie Requirements

The merchandising team queries revenue and inventory data in natural language directly from the Databricks analytics interface, with no code. This is enabled by Databricks AI/BI Genie, which requires:

- Gold layer tables registered in Unity Catalog with descriptive column-level comments (already implemented: every Gold table column has a business-readable comment).
- A Metric View (metricview_inventory) that defines pre-approved business measures (Slow Product Count, Avg Days Since Last Sale, Total Sales) and dimensions (Region, Brand, Category) so Genie produces deterministic, governed results.
- SQL Warehouse as the compute endpoint—Genie translates natural language to SQL and executes against the warehouse, not the pipeline cluster.

AI Output Persistence

All AI-generated outputs persist as Delta tables in the Gold layer:

- ai_business_insights: stores LLM-generated summaries (revenue performance, vendor return rates, slow-moving inventory, quarantine explanations) with kpi_json for structured data and llm_check/llm_check_detail for quality validation of the AI output itself.
- flagged_return_customers: stores investigation briefs alongside the anomaly scoring data. Each brief includes the scored_at timestamp, the rules_violated field, and the LLM quality check.

Both tables are accessible to business teams via SQL Warehouse dashboards and AI/BI Genie, with row-level permissions governed by Unity Catalog.

5. Generative AI Integration

Model and access pattern

All four use cases call the same foundation model: databricks-gpt-oss-20b via the Databricks AI Gateway using the OpenAI SDK pointed at the workspace endpoint. No external API keys are stored anywhere in the codebase. The session token is retrieved at runtime from dbutils.notebook.entry_point. UC4 additionally uses ai_query(), Databricks' SQL-native function that calls the same model directly inside a SELECT statement with no Python client required.

Overview of Usecases

Use Case	Business Team	What Goes In	What Comes Out	Output Table
Data Quality Reporter	Finance (audit prep)	silver.*_quarantine tables grouped by entity + issue_type	3-section audit explanation: What happened, why it cannot be accepted, what is at risk	gold.dq_audit_report
Returns Fraud Investigator	Returns team (40–60 cases/day)	Scored customer profiles from 8-rule anomaly engine (fact_returns, fact_orders, dim_customers)	Per-customer investigation brief: anomaly score, rules violated, innocent explanations, verification steps	gold.flagged_return_customers
Product Intelligence (RAG)	Merchandising team	1,774 natural language product documents embedded in FAISS index	Grounded answer from retrieved documents only — explicit fallback if answer not in index	gold.rag_query_history
Executive Intelligence	Leadership	Aggregated KPIs from Gold MVs (never raw rows)	4–6 sentence executive synthesis: alarming / expected / opportunity + recommended action	gold.ai_business_insights

UC1: AI Data Quality Reporter

The LLM translates Silver quarantine data into plain-English audit explanations for the finance team. Rejections are grouped by entity and issue type first , one LLM call per issue group, not one per rejected row. This produces consistent explanations and avoids hitting context limits.

Source tables	Six Silver quarantine tables: customers_quarantine, orders_quarantine, transactions_quarantine, returns_quarantine, products_quarantine, vendors_quarantine
Aggregation	Grouped by entity + issue_type. Collects: rejected count, up to 3 sample _failure_reason values (actual bad values from the data), up to 4 source file paths shortened to filename (Region N) format, earliest and latest failure timestamps.
Business impact injection	A BUSINESS_IMPACT dictionary maps every (entity, issue_type) pair to the specific report or figure at risk. This is injected into the prompt so the model cites real reports (e.g. 'the monthly revenue trend report') rather than inventing generic ones.
Prompt structure	Three labelled sections enforced in the prompt and validated in code:WHAT IS THE PROBLEM , names the field, count, and actual bad valueWHY THIS RECORD TYPE CANNOT BE ACCEPTED , names what breaks downstreamWHAT IS AT RISK , names the specific report and the team that must act

Output	Plain-English audit explanation. No stack traces, no row-level data, no SQL. Written for a finance director with no technical background.
Output table	globalmart.gold.dq_audit_report , 10 rows this run, all llm_check = PASS
Temperature	0.3 , low, for factual consistency

UC2: Returns Fraud Investigator

The LLM explains fraud signals , it does not detect them. A deterministic 8-rule scoring engine runs first. Only customers scoring 40 or above out of 100 are sent to the LLM for an investigation brief.

Scoring engine(pre-LLM)	8 rules applied to fact_returns, fact_orders, dim_customers. Rules: high return volume, high refund value, high order return rate, refund inflation, return before delivery, duplicate order reason, same day ship return, no matching order. Weights sum to 100. Flag threshold = 40 pts.
What goes in	The customer's actual scored data: total returns, total orders, order return rate, total refund value, total refund inflation, largest single inflation, return reasons used, which rules fired and by how many points, customer segment and region. All figures are real , no generic placeholders.
Prompt structure	Three specific questions:1. What specific numbers make this case suspicious , cite actual figures2. What innocent explanations might exist3. What the returns team should check first , be specific and actionable
Output	3-4 sentence investigation brief per flagged customer. Plain prose, no bullet points.
Output table	globalmart.gold.flagged_return_customers , one row per flagged customer with anomaly_score, rules_violated (pipe-delimited list), investigation_brief, llm_check
Temperature	0.3

UC3: Product Intelligence Assistant (RAG)

A two-stage system. Stage 1: local embedding model converts 1,774 Gold product rows into natural language documents and indexes them in FAISS. Stage 2: the LLM answers questions using only the retrieved documents , it must say so explicitly if the answer is not in the index.

Document construction	Each dim_products row is enriched by joining dim_vendors, mv_slow_moving_products, mv_return_rate_by_vendor, and mv_revenue_by_region. Each row becomes a natural language paragraph covering: product ID, name, brand, category, manufacturer, vendor name and ID, days since last sale, slow-moving regions, vendor return rate and total

	returns, region availability, and regional sales/profit/order count from mv_revenue_by_region.
Why conversion is required	FAISS indexes dense vector embeddings , it cannot embed structured rows directly. The embedding model all-MiniLM-L6-v2 was trained on natural language. A question like 'which products are slow-moving in the West region?' produces a query vector that sits semantically close to documents containing that exact phrase. Without prose conversion, retrieval fails entirely.
Embedding model	sentence-transformers/all-MiniLM-L6-v2 , run locally on the cluster, no external API. Produces 384-dimensional vectors. Index: FAISS IndexFlatL2 (exact L2 nearest-neighbour, no approximation). Persisted to /Volumes/globalmart/gold/rag_artifacts/ , survives cluster restarts, rebuilt only when product count changes.
Retrieval	top_k = 5. Query is embedded using the same model. FAISS returns the 5 documents with smallest L2 distance. For 'which vendor has the highest return rate?' , distances ranged 0.93 to 0.98, a tight cluster confirming strong semantic match.
LLM prompt	System message: answer ONLY from provided documents. If answer absent, say 'I could not find that information in the current product catalog.' Never invent product names, vendor names, or return rates.User prompt: 5 retrieved documents as labelled context + question.
Output	Grounded answer citing specific product IDs, vendor names, return rates, and regional revenue from the retrieved documents. Out-of-scope question 7 ('which customers purchased the most?') correctly returned the fallback , model did not hallucinate.
Output table	globalmart.gold.rag_query_history , 7 rows this run, all llm_check = PASS
Temperature	0.1 , lowest of all four use cases, for maximum factual grounding

UC4: Executive Business Intelligence

Three business domains are processed , revenue performance, vendor return rates, and slow-moving inventory. KPIs are aggregated from Gold Materialized Views before the LLM is called. The model never sees raw rows.

Source tables(aggregated first)	mv_revenue_by_region → revenue KPIs per region with MoM delta and profit marginmv_return_rate_by_vendor → return rate per vendor, total refund amount (rate already a %, not decimal)mv_slow_moving_products → slow product count per region, avg days stagnant, exposure ($)
Region normalisation	Region abbreviations (W, S, E, N) are mapped to full names (West, South, East, North) before any KPI is computed or passed to the LLM. Applied via Spark column expression in Python calls and via CASE WHEN inside the ai_query() SQL.

Prompt structure(Python calls)	Each domain prompt forces four specific answers:1. Is the picture ALARMING, EXPECTED, or AN OPPORTUNITY , justify in one sentence2. Which regions or vendors are driving the gap and why3. Single most likely root cause for declining areas4. What leadership should prioritise in the next 30 days
Outputs this run	Revenue: ALARMING , all regions showing ~95% MoM decline, most likely post-holiday demand lossVendors: ALARMING , all four vendors above 20% benchmark, fleet average 50%, total refund exposure >$530KInventory: ALARMING , 5,140 products, West worst at 2,932 avg days stagnant, $56M exposure
Output table	globalmart.gold.ai_business_insights , 3 rows, one per domain. Each row stores kpi_json (the exact aggregated data passed to the LLM) alongside the ai_summary for full auditability.
Temperature	0.4 , slightly higher to allow natural narrative variation across the three summaries

ai_query() demonstrations , SQL-native LLM calls

UC4 also demonstrates ai_query(), which calls databricks-gpt-oss-20b directly inside a SELECT statement. On this workspace the function returns a plain string , no JSON extraction wrapper required.

Demo	Input (CONCAT prompt)	Output
Demo 1 Vendor return rates	'Vendor ' + vendor_name + ' has a return rate of ' + return_rate_pct + '%. The industry benchmark is 20%. State whether ACCEPTABLE / CONCERNING / CRITICAL, name the most likely cause, and state one action.'	One sentence per vendor , e.g. Johnsons Logistics at 56.1% is CRITICAL, most likely product quality failure; recommend renegotiation.
Demo 2 Revenue by region	'The [region] region generated $[sales] in revenue and $[profit] in profit across [orders] orders. State whether ALARMING / EXPECTED / AN OPPORTUNITY, give the most likely reason, and name one action.'	One sentence per region , e.g. Central is ALARMING given negative profit margin; leadership should audit cost structure within 30 days.

Quality validation - all use cases

Every LLM output is validated before writing to Delta. If validation fails, an automatic retry fires with a longer wait. If the retry passes, the PASS result replaces the original before writing. If the

retry still fails, the record is written as REVIEW for human audit. On this run all outputs across all four use cases returned llm_check = PASS.

Use Case	Too Short	LLM Failed	Missing Section	Retry Behaviour
UC1DQ Reporter	< 30 words	Yes	All 3 labels required	Auto-retry with 5s wait. PASS result replaces original before Delta write.
UC2 Fraud Investigator	< 30 words	Yes	No	Auto-retry with 5s wait.
UC3 RAG Assistant	< 10 words	Yes	No	Not-found answer is valid , model is expected to say so if answer is absent from docs.
UC4 Exec Intelligence	< 40 words	Yes	No	Auto-retry with 5s wait. PASS result replaces original before Delta write.

Design principles

- Aggregate First: Use KPIs and summaries instead of raw rows to stay within context limits and ensure consistency.
- Rules Detect, LLM Explains: Fraud detection is deterministic; the LLM only writes briefs and makes no decisions.
- Ground with Real Data: Inject actual field names, counts, and impact statements to prevent generic outputs.
- Validate Every Output: All AI tables must pass llm_check before reaching business users.
- Auditability: Store kpi_json and rules_violated with every response so analysts can verify the source data.
- No External Dependencies: Use local embeddings and internal gateways so no data leaves the workspace.

5. Data Quality Strategy

Quality enforcement across the GlobalMart platform is proactive, dynamic, and fully integrated into the pipeline. Rather than discovering data anomalies during quarterly audits or dashboard failures, data is continuously evaluated as it flows from raw ingestion to business-level aggregations.

Our expectation strategy follows two fundamental principles:

1. **No record is ever permanently lost.** Every record that fails any expectation — whether critical or advisory — is captured in a quarantine table for that specific entity. Critical failures are

removed from the main analytical tables. Advisory failures remain in the main table but are also logged in quarantine for visibility and tracking.

2. **Severity-based triage.** Each expectation is tagged with a severity level (CRITICAL or WARNING) that determines the pipeline's outcome and guides downstream investigation by the finance, returns, and merchandising teams.

5.1 Quality Enforcement Across Layers

Quality checks are distributed across four distinct checkpoints to ensure structural errors are caught early, while complex business anomalies are flagged after the data is fully unified.

Layer	Check Type	Mechanism	Outcome / Quarantine Action
Pre-Bronze	Contract Validation	schema_validation_bronze evaluates min_cols and key_variants.	**WARN:** Flags incoming Raw streams missing fundamental identifiers or minimum column thresholds before Bronze processing begins.
Silver	Harmonization & Integrity	@dp.expect_or_drop (Critical) and @dp.expect (Warning) based on config metadata.	**QUARANTINE:** Criticals (e.g., null Primary Keys) are dropped from the main flow. Warnings (e.g., missing optional fields like email) pass through. Both are routed to Silver _quarantine tables.
MDM	Entity Safeguards	Multi-factor identity fingerprinting: email primary, name+postal_code fallback. Source priority ranking.	**RESOLVE:** Prevents duplicate keys and overlapping records by merging conflicting cross-regional data into a single Golden Record.
Gold	Business Logic DQ	@dp.expect applied to Fact tables and Materialized Views.	**QUARANTINE:** Flags advanced business anomalies (e.g., "Orphan Customers" missing from dimensions, "Returns Fraud" where refunds > 5x sales). Failures are isolated in Gold quarantine tables.

5.2 Integration with the Broader Architecture

The Spark Declarative Pipeline (SDP) is designed to **never halt** upon encountering dirty data. This ensures the merchandising and finance dashboards are always populated with the latest valid data, while dirty data is safely siphoned off for review.

Here is how the DQ strategy integrates with the rest of the architecture:

- **Seamless Harmonization:** Because the Silver layer utilizes an LLM Schema Mapper to resolve column name variations (e.g., cust_id vs. customer_identifier), the DQ engine rarely trips on simple schema drift. It focuses purely on actual data fidelity.
- **Automated Audit Trails:** When a record is quarantined, it is automatically enriched with audit lineage columns: _expectation_name, _severity, _issue_type, _failure_reason, and a unique _run_id.
- **Gen AI Intelligence Integration:** These structured quarantine tables (e.g., silver.customers_quarantine, gold.fact_returns_quarantine) serve as the direct input for the Gen AI Intelligence Layer. Instead of business users parsing Python logs, the Gen AI reads the quarantine metadata and synthesizes plain-English executive audit briefs, detailing exactly what failed, which regional file it came from, and the business impact.

Quarantine Transparency

Every quarantined record is paired with an AI-generated explanation stored in ai_business_insights. Each explanation follows a three-part structure: WHAT HAPPENED (rule and count), WHERE IT CAME FROM (exact source file path), and BUSINESS RISK AND ACTION (who must act and by when). This ensures the finance team can investigate quarantined records without reading Python code or pipeline logs.

6. Executive Technical Summary

The GlobalMart data platform represents a paradigm shift from fragmented, manual data silos to an intelligent, automated, and governed data ecosystem. By combining Databricks' Medallion Architecture, Spark Declarative Pipelines (SDP), and Generative AI, this architecture directly resolves the three critical business mandates while establishing a highly scalable foundation for future acquisitions.

6.1 Resolution of Core Business Mandates

1. Revenue Audit Consistency (Solving the 9% Overstatement) The pipeline resolves historical revenue overstatements through an innovative Master Data Management (MDM) layer. Instead of relying on disparate regional primary keys, the pipeline employs **MDM Engine**(evaluating combinations of email, name, and postal codes) to map overlapping regional accounts to a single "Golden Record." Furthermore, Unity Catalog provides absolute column-level lineage, allowing auditors to trace any metric in the mv_revenue_by_region view back to its exact source file and timestamp.

2. Returns Fraud Mitigation (Stopping the $2.3M Leakage) Fraud detection shifts from a reactive, multi-week manual process to a proactive, automated pipeline. The Gold layer harmonizes return events with original sales line items using a highly performant **Array-Sort**

Lambda function, bypassing expensive window operations on streaming data. A deterministic 8-rule scoring engine continuously evaluates this unified data (mv_customer_return_history, mv_return_rate_by_vendor) to flag anomalies. Generative AI then processes these signals to output pre-written, plain-English investigation briefs (flagged_return_customers), drastically reducing investigator triage time.

3. Eliminating Inventory Blindspots (Recovering $2.1M Exposure) Slow-moving inventory is no longer obscured by regional data silos. The pipeline automatically calculates velocity metrics (days-since-last-sale) in mv_slow_moving_products. To make this instantly accessible, the platform leverages a **Retrieval-Augmented Generation (RAG)** architecture. 1,774 product records—enriched with sales, returns, and vendor metadata—are embedded via a local all-MiniLM-L6-v2 model into a FAISS index. This powers Databricks AI/BI Genie, enabling the merchandising team to ask natural language questions (e.g., *"Which products are slow-moving in the West region?"*) and receive deterministic, grounded answers without writing SQL.

6.2 Key Architectural Innovations

Beyond solving the immediate business problems, the architecture introduces several advanced technical patterns that ensure long-term resilience:

- **LLM-Driven Schema Discovery:** The Silver layer eliminates brittle, hardcoded ETL scripts. Using Databricks Mosaic AI (databricks-gpt-oss-20b), the pipeline dynamically profiles incoming Bronze data. It auto-corrects severe structural anomalies (such as the Region 4 City/State header swap) and standardizes currencies and abbreviations via semantic pattern recognition, falling back to static configurations only if necessary.
- **"No Data Lost" Quarantine Pattern:** Data Quality is strictly enforced using SDP expectations (@dp.expect_or_drop, @dp.expect). Rather than failing the pipeline or silently dropping non-critical anomalies, dirty data is systematically diverted to entity-specific _quarantine tables.
- **AI-Synthesized Observability:** The platform does not expect business users to read Python logs. The AI Data Quality Reporter (UC1) autonomously reads the quarantine tables and generates structured, plain-English audit reports (dq_audit_report) that explain *what* failed, *where* it came from, and the resulting *business risk*.
- **Compute Isolation and Concurrency:** Heavy data transformations are strictly bound to pipeline clusters, while all business-facing dashboards, Metric Views, and AI/BI Genie workspaces are served via Serverless SQL Warehouses. This guarantees sub-second query performance for 30+ concurrent users, powered by the Photon engine, without interfering with backend ETL loads.

Conclusion:

The 4Ceers GlobalMart architecture delivers a zero-maintenance ingestion engine,

mathematically sound financial reconciliations, and an intuitive AI-driven interface. It transitions GlobalMart from a state of data reactivity and exposure to a state of complete observability, governance, and proactive intelligence.